To handle state persistence and spaced repetition logic purely on the client side—without violating the constraint of a buildless, CDN-served React prototype—you have to rely entirely on browser-native storage APIs.

Since you cannot query a backend server to retrieve a user's mistake history, the frontend application state must act as its own self-contained database. Here are the three most viable architectural solutions for this environment:

1. The localStorage JSON Blob (The MVP Approach)

This is the most straightforward method for a buildless React application. You serialize the user's progress and mistake history into a single JSON object and write it to the browser's local storage.

- **The Logic:** You maintain a central React state object containing unlocked_stages, current_xp, and a mistake_ledger. The ledger is simply an array of objects recording the question_id, stage_id, and failure_count.
- **Implementation:** Use a custom useEffect hook to synchronize this state object with localStorage every time the user completes an exercise or makes a mistake. When a cumulative stage test is generated, your React component parses the JSON, reads the mistake_ledger, and dynamically pulls those specific question IDs from your static content files to populate the test.
- **Pros & Cons:** It is incredibly easy to implement via a single HTML file without a build step. However, localStorage is synchronous, meaning large read/write operations could theoretically block the main thread and cause UI stutter, though this is unlikely given the small data payloads of text-based questions.

2. IndexedDB via CDN Wrapper (The Robust Approach)

If the mistake_ledger grows too large, or if you want to run more complex queries (e.g., "fetch all failed passive voice questions from the last 48 hours"), localStorage becomes inefficient.

- **The Logic:** IndexedDB is an asynchronous, transactional database built directly into the browser. It handles structured data much better and doesn't block the main thread.
- **Implementation:** Raw IndexedDB syntax is notoriously clunky. Since you are using cdnjs.cloudflare.com for dependencies, you can pull in a lightweight wrapper library like localForage via a <script> tag. This gives you a simple, Promise-based API (using async/await) to read and write mistake logs exactly as you would with a NoSQL backend.

3. The "Save State" Export (Addressing Volatility)

The biggest risk of client-side storage is that if a user clears their browser cache or switches from their phone to their laptop, their entire learning history is destroyed. Your requirements specify that progress must not be lost.

- **The Logic:** Implement a manual "Export Save" and "Import Save" feature.
- **Implementation:** Allow the user to download their localStorage state as a lightweight encrypted string or a raw .json file. When they return or switch devices, they can upload that file. Your React app reads the file, hydrates the local state, and updates localStorage.

It acts like a video game save file, empowering the user to own their data before the production phase introduces a real database.

For a Version 1.0 prototype running entirely in the browser, Solution 1 combined with Solution 3 provides the best balance of speed and reliability without complicating your architecture.

As you think about the code structure for this in your React prototype, how do you plan to handle the static content itself (the lesson texts, questions, and Sinhala explanations)? Will you hardcode them directly into the React components, or will you fetch them asynchronously from a separate static JSON file to keep the application logic clean?